16-Channel GPIO Expander Evaluation Board

1 GENERAL DESCRIPTION

This user's guide describes the features and functionality of the **APIO16-EVB**, named as such for its compatibilty with the TSSOP-28 AP (LEO) and AF (GEO) family of devices. Both **APIO16** and **AFIO16** devices are radiation hardened by design, level translating, I^{2}C, SMBUS, SPI integrated IO expanders that are ideally suited for space and other applications requiring high reliability and radiation tolerance for both Single Event Effects (SEE) and Total Ionizing Dose (TID).

The **APIO16-EVB** allows maximum flexibility for evaluation using a variety of host controllers, and comes with the **APIO16** device installed at U1.
Examples provided in this guide include SPI and I^{2}C connections to Raspberry Pi 4b and Total Phase Aardvark/Cheetah adapters.

The **APIO16** is a member of the Apogee Semiconductor APIO16 interface family operating across a voltage supply range of **1.4 V to 5.5 V**.

For more information refer to the APIO16 product page.

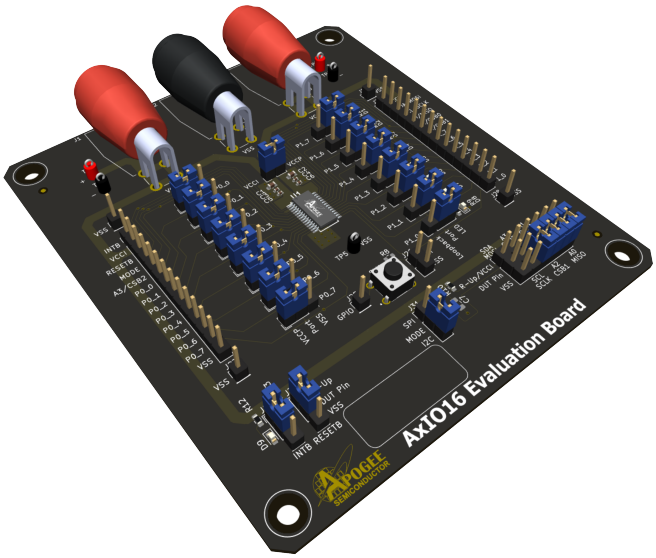

Figure 1: Rendered Image of the APIO16-EVB

1.1 FEATURES

- 16 bits of GPIOs available on two 8-bit ports, Port 0 (P0) and Port 1 (P1)
- Protocol selection jumper (J31) for SPI or I^{2}C operation modes
- Per-bit configurable bias jumpers on Port 0 for VSS or VCCP connection
- Per-bit configurable jumpers on Port 1 for either onboard LED indicators or loopback connections to Port 0
- Configurable I^{2}C address jumpers to support addresses from 0x20 to 0x2F (16 unique addresses on same bus)
- Option to populate capacitors on SDA and SCL lines for testing various I^{2}C capacitive bus loadings in the system
- Selectable onboard pullup resistors for SDA and SCL I^{2}C lines with configurable jumpers.
- INTB signal LED indicator for visual monitoring of input port state changes, with optional host controller connection on header
- Single momentary push button that can be used for GPIO-triggered host events
- Pin headers (J6 and J7) for all device signals providing easy access for testing and evaluation
- Auxiliary header for address and protocol pins to allow custom plug in cable connectivity (J32).
- Configurable jumper to join VCCP and VCCI to simplify single supply usage (J8)

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

CONTENTS

1 General Description **1**
1.1 Features 1

2 IMPORTANT NOTICE **3**

3 Overview **4**

4 Hardware Description **4**
4.1 Power 5
4.2 Breakout Headers 5
4.3 Communications Header 5
4.3.1 High-Speed SPI Configuration . . . 6
4.4 Jumpers 6

5 Port Configuration Headers **8**
5.1 P0 Configuration Header 8
5.2 P1 Configuration Header 8

6 INTB and RESETB **8**
6.1 INTB Header (J29) 8
6.2 RESETb Header (J30) 8
6.3 Push Button Interface (SW1) 8

7 Operation of the APIO16-EVB **9**
7.1 Important Configuration Notes 9
7.2 I2C Mode 9
7.3 I2C Configuration 9
7.4 Implementation Examples 9
7.4.1 Python Implementation 9
7.4.2 Total Phase Aardvark Implementation 10
7.5 SPI Mode 11
7.5.1 SPI Command Format 11
7.5.2 Python Implementation 11
7.5.3 Total Phase Aardvark Implementation 13

8 Hardware Design **14**
8.1 Schematic 14
8.2 Bill of Materials 15

9 Revision History **16**

10 Legal **17**

LIST OF FIGURES

1 Rendered Image of the APIO16-EVB 1
2 EVB Features Annotated (Top View) 4
3 Overview Schematic 14

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

2 IMPORTANT NOTICE

The evaluation kit is intended for engineering development or evaluation purposes only. It is supplied with a sample integrated circuit mounted on a PCB, allowing easy access to inputs, outputs, and power connections. The evaluation board may be used with any development system or other I/O source by connecting it to a host controller or protocol-generating device.

The evaluation board is not a reference design and is not intended to serve as a complete circuit solution for any specific function or application. The exposed circuitry may be vulnerable to environmental damage. Users are responsible for implementing appropriate precautions, including electrostatic discharge (ESD) protection, to ensure safe handling and reliable operation.

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

3 OVERVIEW

Apogee Semiconductor's AP (LEO) and AF (LEO) families of radiation hardened logic, level translators, and RelBridge™ devices are focused on making the frontier of space more accessible by bridging the technology gap between commercial and high-reliability technologies at a lower cost with innovative solutions. The **APIO16-EVB** allows functional and performance validation of the AP and AF families in 28pin TSSOP packages.

The current list of devices that are upcoming, being sampled, or released, can be seen at https://apogeesemi.com/products/.

4 HARDWARE DESCRIPTION

The **APIO16-EVB** evaluation board enables comprehensive testing and evaluation of Apogee Semiconductor's 16 Channel GPIO Expander **APIO16**. This versatile platform offers access to many device features, with configurable I/O, power supply options, and communication interfaces.

The figure below shows the top side of the board with all components and default jumpers settings. Key features mentioned previously are highlighted for reference.

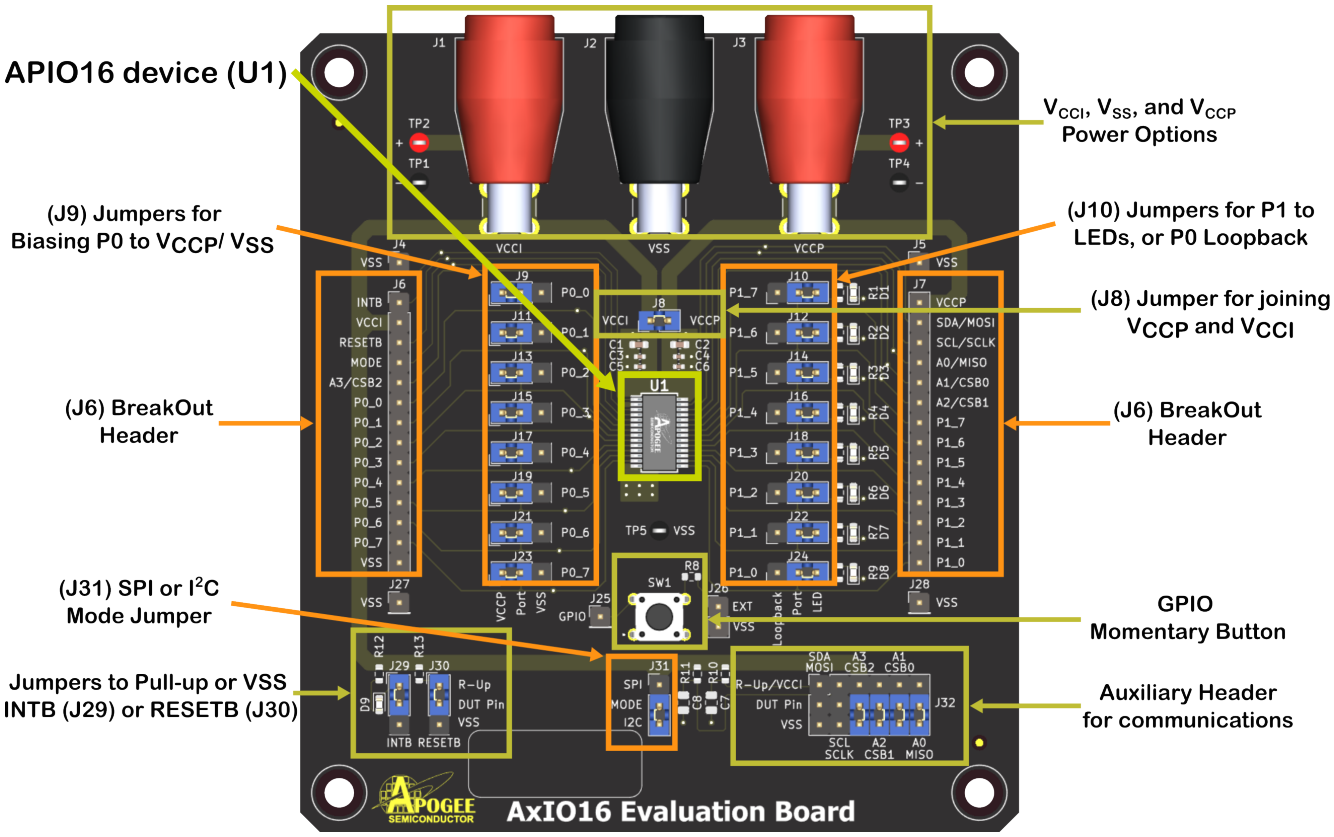

Figure 2: EVB Features Annotated (Top View)

4.1 POWER

The **APIO16-EVB** board can be powered using either a single supply or dual supply configuration through the onboard banana jacks and optional shorting jumper J8. The default configuration is with jumper J8 populated connecting V_{CCP} to V_{CCI} for single supply operation.

When operating with a dual supply voltage, the shorting jumper J8 should be removed to power from a separate V_{CCP} and V_{CCI} supply.

Below is a list of the power components on the **APIO16-EVB**:

- Shorting Jumper (J8) - Connects V_{CCP} to V_{CCI} for single supply operation
- Interface supply (V_{CCI}) - Available via Banana Jack (J1) or Test Point (TP2)
- Port supply (V_{CCP}) - Available via Banana Jack (J3) or Test Point (TP3)
- Board ground (V_{SS}) - Available via Banana Jack (J2), Test Points (TP1, TP3), or any V_{SS} header pins

4.2 BREAKOUT HEADERS

Two breakout headers (J6 and J7) are provided on the top side with 100mil pitch spacing:

- J6: Pins 1-14 of the device
- J7: Pins 15-28 of the device

These headers provide direct 1:1 access to all device pins for prototyping purposes. refer to datasheet for pinout of the device.

4.3 COMMUNICATIONS HEADER

The J32 header is a 3x6 pin array that provides flexible signal connectivity for SPI or I^2C control. The header is organized in 6 columns (A through F), where each column contains 3 pins with the following connections:

- Top pin (Pin1): Connection to V_{CCI} (Interface Power Supply)
- Middle pin (Pin2): Connection to DUT (Device Under Test)
- Bottom pin (Pin3): Connection to V_{SS} (Ground)

This arrangement allows:

- Pull up to V_{CCI} by connecting to the top pin
- Connect to the DUT directly through the middle pin
- Pull down to ground by connecting to the bottom pin
- Note: J32A-Pin2 and J32B-Pin2 have additional 4.7kΩ pull-up resistors to V_{CCI}

The board provides flexibility for:

- Configuring I^2C target address
- Selecting which CSBx signal is used for SPI transactions
- Custom signal routing for specific applications

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

4.3.1 High-Speed SPI Configuration

For high-speed SPI applications, signal integrity can be improved by removing the zero-ohm resistors (R14-R19) on the bottom side of the board. The following table shows the signal connections and their corresponding resistors:

Resistor	Signal	Comunication Header (J32) Signal Pin	Description
R14	MOSI	J32A-Pin2	Main Out Secondary In
R15	SCLK	J32B-Pin2	Serial Clock
R16	CSB2	J32C-Pin2	Chip Select 2
R17	CSB1	J32D-Pin2	Chip Select 1
R18	CSB0	J32E-Pin2	Chip Select 0
R19	MISO	J32F-Pin2	Main In Secondary Out

Table 1: APIO16-EVB Communication Header (J32)

Note: When these resistors are removed for high-speed operation, the signals must be provided through the breakout headers J6 and J7 instead of J32.

4.4 JUMPERS

The APIO16 board contains a number of jumpers that are used to configure Port 0 and Port 1 of the device. The Table below contains the descriptions of all jumpers and their default configuration:

Reference Designator	Type	Name	Default Config.	Description
J8	Power	V_{CCP}/V_{CCI} Short	1-2	For Single supply usage
J9	Signal	P0_0, Port 0 bit 0	1-2	3x1 pin headers for Port 0 bits. Can be configured to bias to V_{SS} (1-2) or V_{CCP} (2-3)
J11	Signal	P0_1 (Port 0 bit 1)	1-2	
J13	Signal	P0_2 (Port 0 bit 2)	1-2	
J15	Signal	P0_3 (Port 0 bit 3)	1-2	
J17	Signal	P0_4 (Port 0 bit 4)	1-2	
J19	Signal	P0_5 (Port 0 bit 5)	1-2	
J21	Signal	P0_6 (Port 0 bit 6)	1-2	
J23	Signal	P0_7 (Port 0 bit 7)	1-2	
J10	Signal	P1_7 (Port 1 bit 7)	2-3	3x1 pin headers for Port 1 bits. Can be configured to enable loopback operation to Port 0 (1-2) or to enable LED (2-3)
J12	Signal	P1_6 (Port 1 bit 6)	2-3	
J14	Signal	P1_5 (Port 1 bit 5)	2-3	
J16	Signal	P1_4 (Port 1 bit 4)	2-3	
J18	Signal	P1_3 (Port 1 bit 3)	2-3	
J20	Signal	P1_2 (Port 1 bit 2)	2-3	
J22	Signal	P1_1 (Port 1 bit 1)	2-3	
J24	Signal	P1_0 (Port 1 bit 0)	2-3	

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

Reference Designator	Type	Name	Default Config.	Description
J25	Signal	GPIO	1	Shorts to V_{SS} when SW8 pressed (interrupt to host) Optional pullup via R8 (not populated)
J29	Signal	INTB	1-2	Connect to LED (1-2) or short to V_{SS} (2-3), or connect to host to monitor for input state changes
J30	Signal	RESETB	1-2	Connect pullup to V_{CCI} (1-2), short to V_{SS} (2-3), or be driven by host
J31	Signal	MODE	2-3	Select I^2C (pin 2-3) or SPI (pin 1-2) mode
J32	Signals	Aux Header	N/A (A to B) 2-3 (C to F)	6x3 header block for I2C/SPI configuration: • Row configuration: – Top: V_{CCI} pullup – Middle: Signal connections – Bottom: V_{SS} • Column usage: – A-B: Interface signals – C-F: I^2C Address selection
J32-A	Signal	SDA/MOSI	N/A	Signal insertion, or V_{CCI} pullup
J32-B	Signal	SCL/SCLK	N/A	Signal insertion, or V_{CCI} pullup
J32-C	Signal	A3/CSB2	2-3	Signal or V_{CCI}/V_{SS} bias. A3=Low for default I^2C address 0x20
J32-D	Signal	A2/CSB1	2-3	Signal or V_{CCI}/V_{SS} bias. A2=Low for default I^2C address 0x20
J32-E	Signal	A1/CSB0	2-3	Signal or V_{CCI}/V_{SS} bias. A1=Low for default I^2C address 0x20
J32-F	Signal	A0/MISO	2-3	Signal or V_{CCI}/V_{SS} bias. A0=Low for default I^2C address 0x20

Table 2: Jumpers Descriptions

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

5 PORT CONFIGURATION HEADERS

5.1 P0 CONFIGURATION HEADER

The Port 0 configuration header is set by default to use P0 as an input, with each pin biased to V_{CCP}. Each port bit can be manually biased to either V_{CCP} or V_{SS} using a jumper. Alternatively, a two-pin signal cable (signal and V_{SS}) can be used to connect any Port 0 pin to an external signal source or destination.

5.2 P1 CONFIGURATION HEADER

The Port 1 configuration header is set by default to use P1 as an output with LEDs connected. Each port bit can be manually configured with a jumper to enable loopback to its corresponding bit on P0, or to connect to an LED for visual indication of signal state.

Note: When operating at minimum V_{CCP} , the LED drivers may not provide enough voltage for illumination. For reliable LED visibility, operate above approximately 2.5V .

6 INTB AND RESETB

6.1 INTB HEADER (J29)

The INTB header (J29) is a 3-pin connector for the interrupt signal:

- Pins 1-2: Connects INTB to LED indicator. LED will illuminate when input port changes state
- Pins 2-3: Routes INTB to host controller for monitoring input state changes

Note: Interrupt detection requires at least one port read after power-up. See datasheet for details.

6.2 RESETB HEADER (J30)

The RESETB header (J30) is a 3-pin connector for reset control:

- Pins 1-2: Connects a 49.9kΩ pull-up to V_{CCI} (recommended)
- Pins 2-3: Allows external reset from host controller

6.3 PUSH BUTTON INTERFACE (SW1)

The push button interface consists of SW1, J25, and J26:

- SW1 connects J25 to V_{SS} when pressed
- Unpopulated resistor to J26 available for optional pull-up or level translation

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

7 OPERATION OF THE APIO16-EVB

7.1 IMPORTANT CONFIGURATION NOTES

The evaluation board does not include internal pullup or pulldown resistors for the ports. To ensure proper operation:

- Do not leave input pins floating
- Configure P1 as outputs when using LED connections
- Be aware that LED and resistor parasitics can pull P1 inputs into intermediate voltage ranges, potentially causing excessive current in input mode

Caution

When using loopback connections from P1 to P0, ensure proper port configuration to prevent shorting P0 to P1 outputs. Incorrect configuration may **damage** the device.

7.2 I²C MODE

Operating the device in I²C mode is straightforward. The APIO16-EVB comes pre-configured for I²C operation with a default address of 0x20. The address can be modified by configuring jumpers on J32C, J32D, J32E, and J32F, allowing selection from sixteen possible addresses by connecting each jumper to either V_{CCI} or V_{SS}. For detailed address configuration options, refer to the device datasheet.

7.3 I2C CONFIGURATION

To configure the board for I²C communication:

- Connect the host controller's SDA and SCL lines to J32A and J32B respectively
- Connect V_{SS} from any available ground point on the board to the host controller
- Connect V_{CCI} and V_{CCP} to appropriate power supplies
- For single supply operation, use the default J8 jumper configuration to connect V_{CCI} and V_{CCP}

Note: This configuration has been successfully tested with both a Raspberry Pi 4B running Python and a Total Phase Aardvark protocol analyzer.

7.4 IMPLEMENTATION EXAMPLES

7.4.1 Python Implementation

The following Python code demonstrates I²C communication using a Raspberry Pi 4B running Raspberry Pi OS. This example assumes P0 is configured with any input pattern and P1 is set to drive LEDs.

```
import smbus
from time import sleep

#  AxIO16 Register Map
#  Reg# : name                : access     : default  (reset)
#  0x00 input port 0          : read only  : xxxxxxxx (unknown)
#  0x01 input port 1          : read only  : xxxxxxxx (unknown)
#  0x02 output port 0         : r/w        : 11111111 (out hi)
#  0x03 output port 1         : r/w        : 11111111 (out hi)
#  0x04 Polarity port 0       : r/w        : 00000000 (no inv)
#  0x05 Polarity port 1       : r/w        : 00000000 (no inv)
```

```
#  0x06 Configuration port 0 : r/w        : 11111111 (input)
#  0x07 Configuration port 1 : r/w        : 11111111 (input)

# Rpi setup/configuration.
# Rpi channel
channel = 1
# EVM I2C address.  A0:A3 = Low
address = 0x20
# set bus definition for Rpi
bus = smbus.SMBus(channel)

# Code to write single byte to Rpi I2C channel
# bus.write_byte_data(address, register, value)

# Code to read single byte to Rpi I2C channel
# val = bus.read_byte_data(address, register)

#Set P0 as input (this is default after power up or reset)
bus.write_byte_data(address, 0x06, 0xff)
#Set P1 as output.  LED jumpers intended to be set on P1 pins.
bus.write_byte_data(address, 0x07, 0x00)

# Read jumper setting on P0
val = bus.read_byte_data(address, 0x00)
print ("P0 read:", val)

# The following while loop toggles LEDs in blinking pattern.
while(True)
       bus.write_byte_data(address, 0x03, 0xaa)
       sleep(delay)
       bus.write_byte_data(address, 0x03, 0x55)
       sleep(delay)
```

7.4.2 Total Phase Aardvark Implementation

The following example demonstrates I²C communication using a Total Phase Aardvark protocol analyzer. This implementation provides an alternative method for testing and debugging the device.

- Configure the Aardvark for communication as described in Section 7.3
- Launch the Total Phase Control Center GUI
- Using Control Center GUI, connect to Aardvark in batch mode
- Load and execute the example script below;

Example: Script for I²C communication using Aardvark

```
<adapter>
<!---->
<!--Script for setup P0 input, P0 output with simple read of P0 and LED output
   to P1-->
<configure i2c="1" spi="0" gpio="0" tpower="0" pullups="0"/>
<i2c_bitrate khz="100"/>
<!--Set P0 to input mode-->
<i2c_write addr="0x20" count="2" nostop="0" ten_bit_addr="0" combined_fmt="0"
   radix="16"> 06 FF </i2c_write>

```

```
<!--Set P1 to output mode-->
<i2c_write addr="0x20" count="2" nostop="0" ten_bit_addr="0" combined_fmt="0"
    radix="16"> 07 00 </i2c_write>

<!--Read P0-->
<i2c_write addr="0x20" count="2" nostop="0" ten_bit_addr="0" combined_fmt="0"
    radix="16"> 00 00 </i2c_write>

<!--Write P1-->
<i2c_write addr="0x20" count="2" nostop="0" ten_bit_addr="0" combined_fmt="0"
    radix="16"> 03 aa </i2c_write>
<sleep ms="100"/>
<i2c_write addr="0x20" count="2" nostop="0" ten_bit_addr="0" combined_fmt="0"
    radix="16"> 03 55 </i2c_write>
</adapter>
```

7.5 SPI MODE

To configure the device for SPI operation:

- Set device to SPI mode by placing a jumper on J31 across pins 1-2
- Connect SCLK, MOSI, MISO, and one CSBx signal (e.g., CSB0 on J7 pin 5) to the host controller.
- Configure the remaining two CSBx signals by setting their respective jumpers on J32 to V_{SS}

Note: All three CSBx signals (CSB0, CSB1, CSB2) are active low. For proper operation, unused CSBx signals must be pulled to V_{SS} via jumpers on J32, while the active CSBx signal connects to the host controller's SS ir CS line.

7.5.1 SPI Command Format

The SPI interface uses a specific command format for register access of the device. While Python SPI libraries handle this automatically, manual formatting is required for Aardvark implementations.

Each SPI transaction uses an 8-bit command byte:

- bit[7:5] Register address (C2, C1, C0)
- bit[4] Operation type Write (1) / Read (0)
- bit[3] Must be 0
- bit[2:0] Don't care (typically 000)

Example: Writing to register 3 comprises of C2, C1, C0 = 111 (binary 111), a Write bit 1, followed by 0XXX. Resulting in command byte 0xF0 (1111 0000). Refer to the datasheet for more information.

7.5.2 Python Implementation

Example: Python code for SPI communication using Raspberry Pi 4b:

```
import spidev
from time import sleep

#  AxIO16 Register Map
#  Reg# : name                : access     : default  (reset)
#  0x00 input port 0          : read only  : xxxxxxxx (unknown)
#  0x01 input port 1          : read only  : xxxxxxxx (unknown)
#  0x02 output port 0         : r/w        : 11111111 (out hi)
```

```
#  0x03 output port 1        : r/w        : 11111111 (out hi)
#  0x04 Polarity port 0      : r/w        : 00000000 (no inv)
#  0x05 Polarity port 1      : r/w        : 00000000 (no inv)
#  0x06 Configuration port 0 : r/w        : 11111111 (input)
#  0x07 Configuration port 1 : r/w        : 11111111 (input)
Rpi setup/configuration.
# Rpi channel
spi = spidev.SpiDev()
spi.open(0,0) # open spi bus 0, device 0
spi.max_speed_hz = 25000000

def SPI_Write(register, value):
    # Create the command byte for the write operation
    # Bits 7:5 = register (0-7)
    # Bit 4 = 1 for write
    # Bit 3 = 0 for read or write operation
    # Bits 2:0 = don't care (set to 000)
    command_byte = (register << 5) | (1 << 4) | (0 << 3)  # Read/Write set to 1, Bit 3 set to 0
    data_byte = value  # Data to write
    #print(register, value, command_byte, data_byte)

    # Send the command byte followed by the value byte
    response = spi.xfer2([command_byte, data_byte])
    return response  # You can handle the response if needed

def SPI_Read(register):
    # Create the command byte for the read operation
    # Bits 7:5 = register (0-7)
    # Bit 4 = 0 for read
    # Bit 3 = 0 for read or write operation
    # Bits 2:0 = don't care (set to 000)
    command_byte = (register << 5) | (0 << 4) | (0 << 3)  # Read/Write set to 0, Bit 3 set to 0

    # Send the command byte and receive the response
    response = spi.xfer2([command_byte, 0])  # Send 0 as dummy data byte for read
    return response[1]  # Return the second byte (data from register)

#Set P0 as input (this is default after power up or reset)
SPI_Write(6, 0xff)
#Set P1 as output.  LED jumpers intended to be set on P1 pins.
SPI_Write(7, 0x00)
# Read jumper setting on P0
val = SPI_Read(0x00)
print ("P0 read:", val)
# The following while loop toggles LEDs in blinking pattern.
while(True)
       SPI_Write( 3, 0xaa)
       sleep(delay)
       SPI_Write( 3, 0x55)
       sleep(delay)
```

7.5.3 Total Phase Aardvark Implementation

Total Phase Aardvark Configuration:

- Connect SCLK, MOSI, MISO, CSB(1, 2,or 3) and V_{SS} to the respective Total Phase header pins (SCLK, MOSI, MISO, SS and GND)
- Launch Total Phase Control Center GUI
- Connect to Aardvark in batch mode
- Load and execute the example script below:

Example: Script for SPI communication using Aardvark

```
<adapter>
<!---->
<!--Script for setup P0 input, P0 output with simple read of P0 and LED output to P1-->
<!--Aardvark max speed for SPI is 8Mhz -->
<configure i2c="0" spi="1" gpio="0" tpower="0" pullups="0"/>
<spi_bitrate khz="8000"/>

<spi_config polarity="rising/falling" phase="sample/setup" bitorder="msb" ss="active_low"/>

<!--F0 write P1 as outputs. 3 address bit (reg 7 111) followed by r/w bit "1", leading to F. Next bit must be 0, followed by 3 don't cares resulting in F0 00 to set output port register 7 to all 0's-->
<!--Set P0 to input mode reg 6 binary - 1101 0000 - D0 -->
<spi_write count="2" radix="16" > D0 FF </spi_write>

<!--Set P1 to output mode reg 7 binary - 1111 0000 - F0-->
<spi_write count="2" radix="16" > F0 00 </spi_write>

<!--Write alternating output pattern to P1-->
<!--70 is write to register 3 binary 0111 0000 - 70 -->
<spi_write count="2" radix="16" > 70 55 </spi_write>
<sleep ms="100"/>
<spi_write count="2" radix="16" > 70 AA </spi_write>
</adapter>
```

8 HARDWARE DESIGN

8.1 SCHEMATIC

Top level pseudo schematic showing default jumper settings.

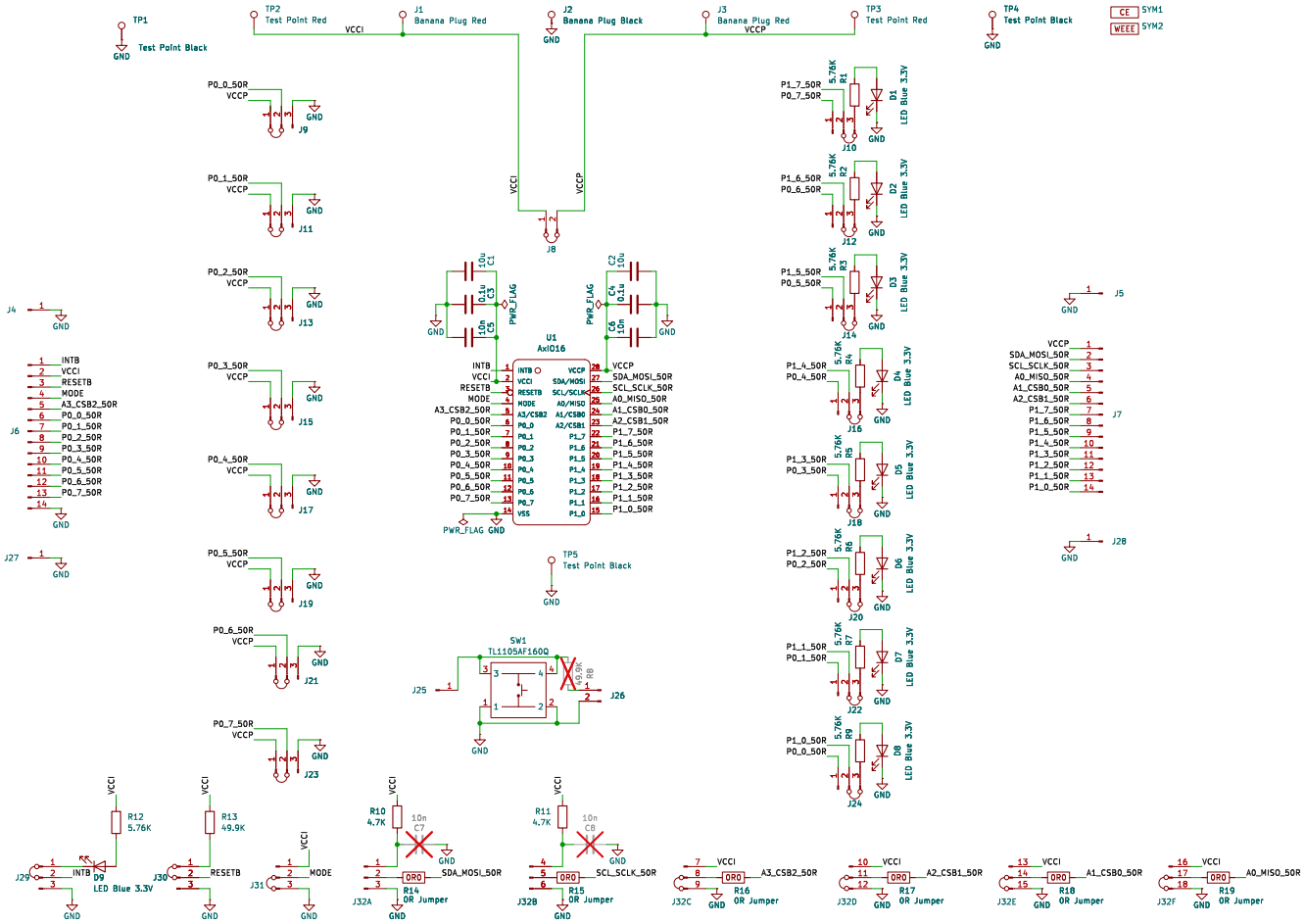

Figure 3: Overview Schematic

8.2 BILL OF MATERIALS

#	Designator	Qty	Manufacturer	Mfg Part #	Description / Value	Type
1	U1	1	Apogee Semiconductor	APIO16	GPIO EXPANDER	SMD
2	SW1	1	E-Switch	TL1105AF160Q	SWITCH TACTILE SPST-NO 0.05A 12V	TH
3	C1,C2	2	Samsung Electro-Mechanics	CL10B106MQ8NRNC	CAP CER 10UF 6.3V X7R 0603	SMD
4	C3,C4	2	Samsung Electro-Mechanics	CL05B104KA5NNNC	CAP CER 0.1UF 25V X7R 0402	SMD
5	C5,C6	2	Samsung Electro-Mechanics	CL05B103KA5NNNC	CAP CER 0.01UF 25V X7R 0402	SMD
6	C7,C8	2	n/a	n/a	n/a	DNP
7	D1-D9	9	SunLED	XZCBD53W-1	LED BLUE CLEAR CHIP SMD	SMD
8	J1,J3	2	Cal Test Electronics	CT3151SP-2	CONN BANANA JACK SOLDER RED	TH
9	J2	1	Cal Test Electronics	CT3151SP-0	CONN BANANA JACK SOLDER BLACK	TH
10	J4,J5,J25,J27,J28	5	Würth Elektronik	61300111121	CONN HEADER VERT 1POS	TH
11	J8,J26	2	Würth Elektronik	61300211121	CONN HEADER VERT 2POS 2.54MM	TH
12	J9-J24, J29-J31	19	Würth Elektronik	61300311121	CONN HEADER VERT 3POS 2.54MM	TH
13	J6, J7	2	Würth Elektronik	61301411121	CONN HEADER VERT 14POS 2.54MM	TH
14	J32	1	Samtec Inc.	TSW-106-07-G-T	CONN HEADER VERT 18POS 2.54MM	TH
15	R1-R7,R9,R12	9	Stackpole Electronics Inc	RMCF0603FT5K76	RES 5.76K OHM 1% 1/10W 0603	SMD
16	R8	1	Stackpole Electronics Inc	RMCF0603FT49K9	RES 49.9K OHM 1% 1/10W 0603	DNP
17	R10, R11	2	Stackpole Electronics Inc	RMCF0603FT4K70	RES 4.7K OHM 1% 1/10W 0603	SMD
18	R13	1	Stackpole Electronics Inc	RMCF0603FT49K9	RES 49.9K OHM 1% 1/10W 0603	SMD
19	R14-R19	6	Vishay Dale	RCL04060000Z0EA	RES SMD 0 OHM 1/4W 0604 WIDE	SMD

Table 3: Bill of Materials

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

9 REVISION HISTORY

REVISION	DESCRIPTION	DATE
A01	Initial Release	2025-05-23

For the latest version of this document, please visit https://www.apogeesemi.com.

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.

10 LEGAL

All product, product specifications and data are subject to change without notice.

Apogee Semiconductor provides technical data (such as datasheets), design resources (including reference designs), reliability data (including performance in radiation environments), application or other design advice, safety information, and other resources **"as is"** and with all faults, and disclaims all warranties, express and implied, including without limitation any implied warranties of merchantability, fitness for a particular purpose or non-infringement of third party intellectual property rights. These resources are intended for skilled engineers with understanding of high reliability and high radiation environments and its complexities.

Apogee Semiconductor is not responsible for: (1) selecting the suitable products for a given application, (2) designing, verifying, validating and testing it, or (3) ensuring that it meets any performance, safety, security, or other requirements. These resources are subject to change without advance notice. The use of these resources is restricted to the development of an application that uses the Apogee Semiconductor products described in them. Other reproduction and display of these resources is prohibited. No license is granted to any other Apogee Semiconductor intellectual property right or to any third-party intellectual property right.

Apogee Semiconductor disclaims responsibility and reserves the right to demand indemnification for any claims, damages, costs, losses, and liabilities arising out of wrongful use of these resources. The products are provided subject to Apogee Semiconductor's Terms of Sale (https://www.apogeesemi.com/terms) or other applicable terms provided in conjunction with applicable products. The provision of these resources does not expand or otherwise alter applicable warranties or warranty disclaimers for Apogee Semiconductor products.

Purchasers of these products acknowledge that they may be subject to and agree to abide by the United States laws and regulations controlling the export of technical data, computer software, electronic hardware and other commodities. The transfer of such items may require a license from the cognizant agency of the U.S. Government.

COPYRIGHT 2025 APOGEE SEMICONDUCTOR INC.